

Tarjeta de referencia

Una hoja. Imprímela a doble cara.

Estructura mínima

```
mi-skill/  
└─ SKILL.md
```

```
---  
description: [qué hace]. Úsala cuando [cuándo usarla].  
---  
  
## Instrucciones  
[qué debe hacer Claude]
```

El nombre del directorio es el comando: `/mi-skill`

Dónde ponerla

ALCANCE	RUTA
Todos tus proyectos	<code>~/.claude/skills/<nombre>/SKILL.md</code>
Este repositorio	<code>.claude/skills/<nombre>/SKILL.md</code>

Precedencia: empresarial › personal › proyecto

La descripción · 4 reglas

1. Tercera persona — Analiza... nunca Te puedo ayudar...
2. Qué hace + cuándo usarla — ambas mitades obligatorias
3. Caso principal al inicio — si se trunca, se trunca por el final
4. Las palabras que tú escribes — no las del manual

Tope: 1,536 caracteres

Frontmatter útil

```
description:           # lo único recomendado  
when_to_use:          # frases disparadoras extra  
allowed-tools:         # preaprueba, NO restringe  
disallowed-tools:     # retira herramientas  
disable-model-invocation: # true = solo tú la invocas  
user-invocable:       # false = solo Claude la invoca  
context: fork         # subagente aislado  
paths:                # globs que limitan activación
```

⚠ `allowed-tools` concede, no restringe. Se limpia con tu siguiente mensaje.

Grados de libertad

GRADO	CUÁNDO	CÓMO SE ESCRIBE
Alta	Varios caminos sirven	Prosa, dirección general
Media	Hay patrón preferido	Plantilla o script con parámetros
Baja	Se puede romper algo	Comando exacto + prohibición de variarlo

Multiarchivo

```
mi-skill/  
├─ SKILL.md           ← procedimiento  
├─ reference.md       ← detalle, solo si hace falta  
└─ scripts/  
    └─ revisar.sh     ← se EJECUTA
```

1. Un solo nivel de referencias — nunca cadenas

2. Los scripts se ejecutan, no se leen
3. Rutas con `${CLAUDE_SKILL_DIR}`

Límite: `SKILL.md` < 500 líneas

Truco anti-permiso:

```
allowed-tools: Bash(${CLAUDE_SKILL_DIR}/scripts/x.sh *)
```

La misma variable en el cuerpo y en la regla → no te pregunta.

Contexto dinámico

```
!`git diff HEAD`  
!`npm run lint || true`
```

⚠ Un comando que falla **aborta toda la invocación**. Usa `|| true` si puede salir con error normal.

Diagnóstico

COMANDO	QUÉ DICE
<code>/skills</code>	Qué hay cargado y de dónde
<code>/doctor</code>	Instalación y costo del listado
<code>/context</code>	Qué ocupa la ventana
<code>claude --debug</code>	Errores de parseo
<code>claude plugin validate .claude/skills</code>	Frontmatter roto en todo el dir

Descarte rápido

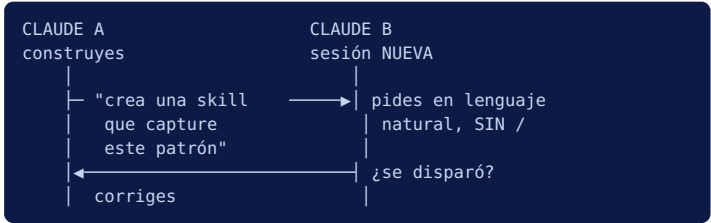
Pregunta: ¿qué skills tienes disponibles?

RESULTADO	PROBLEMA
Aparece con descripción	Redacción
Aparece sin descripción	Frontmatter no parsea
No aparece	Ubicación o visibilidad

Síntomas

SÍNTOMA	CAUSA MÁS PROBABLE
No se activa nunca	Descripción sin las palabras reales, o 1 ^ª /2 ^a persona
Responde a <code>/nombre</code> pero nunca sola	Frontmatter mal formado
Se activa de más	Descripción demasiado amplia
No encuentra un archivo	Ruta absoluta o barra invertida
Salta solicitud de permiso	Falta declarar en <code>allowed-tools</code>
A veces sí, a veces no	Cadena de referencias de 2 niveles
Se aborta sin explicación	Un comando  falló

El ciclo



- No se disparó → arregla la descripción
- Se disparó mal → arregla el cuerpo

Qué capa usar

CONTENIDO	CAPA
Hecho permanente	CLAUDE.md
Procedimiento	Skill
Contexto aislado	Subagente / context: fork
Debe cumplirse siempre	Hook
Sistema externo	MCP

Desempates:

- CLAUDE.md o skill → ¿cada conversación o solo algunas?
- Skill o hook → ¿confío en que el modelo decida?

Antes de compartir

- Descripción en 3ª persona, con qué hace y cuándo
- Nombre del directorio en gerundio
- Grado de libertad declarado y correcto
- < 500 líneas, referencias a un nivel
- allowed-tools pide lo mínimo
- disable-model-invocation si tiene efectos
- Sin secretos, sin datos de cliente, sin caducidad
- Alguien más la corrió sin que le explicaras

Las 7 frases

1. La descripción está siempre en contexto. El cuerpo, no.
2. Si no se dispara, el problema está en la descripción.
3. La pregunta no es cuánto detalle. Es qué tan frágil es la tarea.
4. ¿Hace falta siempre, a veces, o sin excepción?
5. Una skill en el repo es infraestructura del equipo.
6. Las fallas de sintaxis las encuentra el validador. Las descripciones malas, el uso.
7. Solo agrega lo que Claude no sabe ya.